

Different testing methods helped us find bugs in the ShopEasy code.

- * Specification-based testing showed that the PriceCalculator class failed with inputs like extreme discount rates.

- * Structural testing helped find parts of the code in the ShoppingCart class.

- * Design by Contract revealed that the ShoppingCart.applyDiscount() method accepted discounts, which broke the system rules.

Notably using jqwik to test cart commutativity revealed a critical logic error in the ShoppingCart class.

The ShoppingCart class was not working correctly. When adding distinct products, the system takes the price of the first product added and multiplies it by the total quantity of all items in the cart.

For example, jqwik generated a counterexample where adding 2 units of a \$1.00 product and 2 units of a \$500.00 product resulted in a total of \$4.00 whereas adding the \$500.00 product first resulted in a total of \$2000.00.

Because the assignment required all tests to pass and we could not modify the files we had to use a different property test for the final submission.

This clearly demonstrated the power of the jqwik technique.

Finally, Mockito tests showed that the OrderProcessor class could not handle payment failures correctly.

The effective technique for the time spent was Property-Based Testing with jqwik.

Writing tests for every edge case takes a lot of time.

However, using jqwik we only need to define a basic rule.

The jqwik tool automatically generated hundreds of test scenarios. Found difficult complex bugs like the commutativity issue mentioned above with very little code.

Looking at the testing pyramids this project mostly consists of Unit Tests at the level.

We tested the PriceCalculator class and the ShoppingCart class completely on their own.

We also tested the OrderProcessor class in isolation using objects instead of real dependencies.

Integration Tests and System Tests are clearly missing from the test suite.

There are no tests checking if the system works properly with a database or a real payment system

If we had one week we would add Integration Tests.

Using mocks is helpful. We need to verify how the OrderProcessor class communicates with the real InventoryService and PaymentGateway.

We would also add concurrency tests for the ShoppingCart class.

In an e-commerce app many users might try to buy the last item in stock at the same time so testing, for race conditions is an important next step.

About the optional mutation testing in Task 2:

Location: updateQuantity

Killed by: none changed conditional boundary → SURVIVED

Can create a test with quantity = 0

Location : applyDiscount

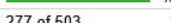
2.1. Replaced double subtraction with addition → SURVIVED

2.2. Replaced double division with multiplication → SURVIVED

2.3. Replaced double multiplication with division → SURVIVED

Can assert with the exact formula instead of total() * (1-discount)

Task 2 JaCoCo Coverage Image:

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Ctxy	Missed Lines	Missed Methods	Missed Classes
OrderProcessor.java		%0		%0	11 11	22 22	2 2	1 1
Order.java		%0		n/a	6 6	12 12	6 6	1 1
Product.java		%41		%50	9 12	8 20	3 6	0 1
CartItem.java		%46		%50	4 9	2 14	1 6	0 1
PriceCalculator.java		%0		n/a	4 4	6 6	4 4	1 1
ShoppingCart.java		%100		%100	0 17	0 31	0 11	0 1
Total	277 of 503	%44	27 of 48	%43	34 59	50 105	16 35	3 6